



**Team
AXIOMATA**

DATA MODEL AND DATABASE

Developed By
Team AXIOMATA

Developed for
Waypoint Group (Pvt) Ltd.

Access to RELAY

1. Overview

Relay stores Waypoint's delivery day in a single PostgreSQL 16 database with 26 tables, defined as SQLAlchemy 2 models and created by one **Alembic migration**. The schema is split into 10 global (reference) tables shared across all workspaces, and 16 workspace tables that carry `workspace_id` for multi-tenant isolation.

2. Entity-relationship diagrams

How to read the diagrams:

An edge label names the foreign-key column.

A table's primary key is `id` unless its box shows another one.

A dotted edge is a reference the code relies on but that has no foreign key.

Columns are listed in section 3.

2.1 Reference data and accounts (global)

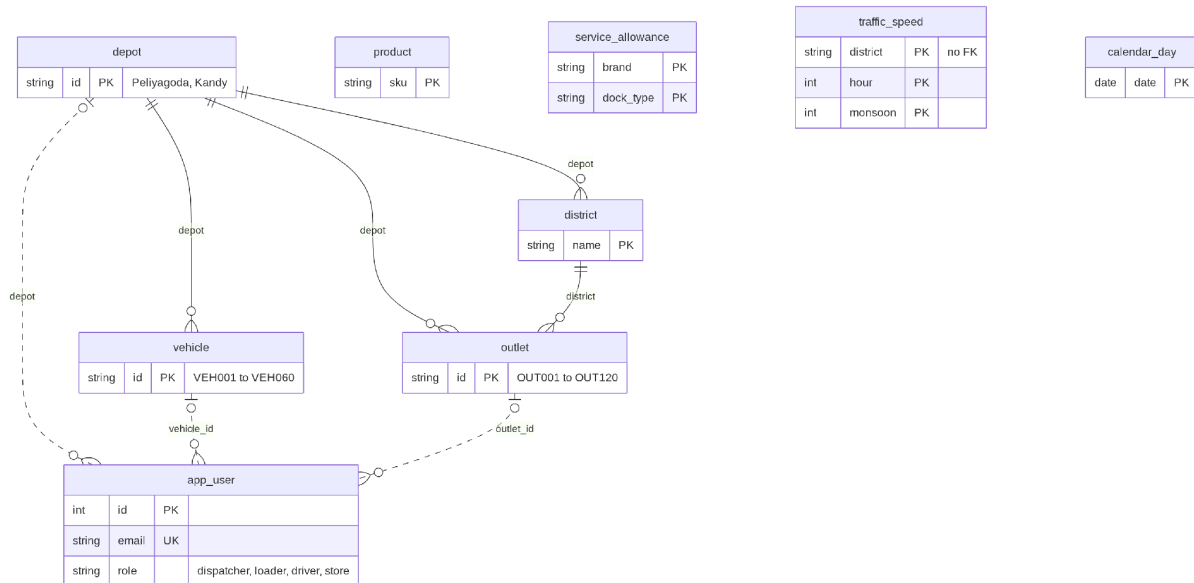


Figure 1: Reference data and accounts (global)

2.2 Planning and execution (workspace tables)

Every table in this diagram except `order_line` also has `workspace_id`. Those edges are left out to keep the diagram readable. `order_origin_order_id` points from a carried-over order back to the original order.

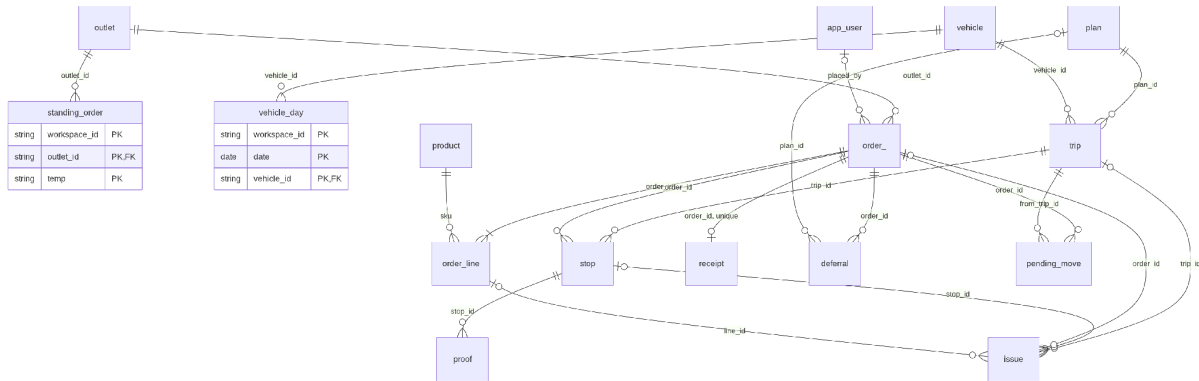


Figure 2: Planning and execution (workspace tables)

2.3 Workspace, notices and the event log

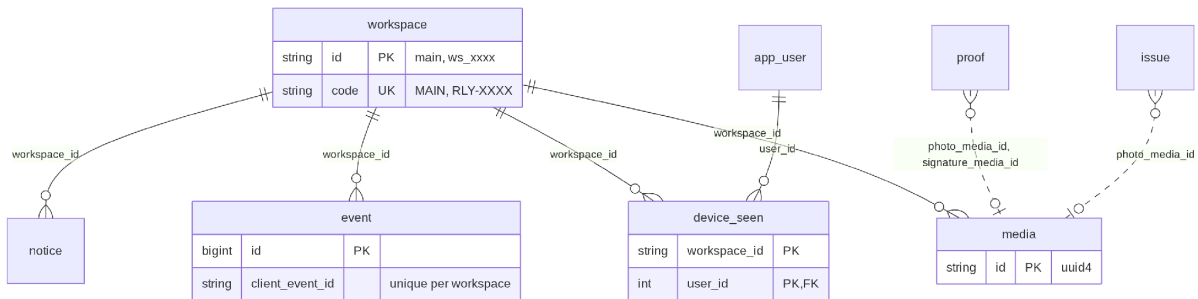


Figure 3: Workspace, notices and the event log

3. Global (Reference) Tables

Table	Rows	Purpose
depot	2	Peliyagoda and Kandy - bays_per_dock, dock count
district	12	Travel times and distances (depot-to-district, inter-stop)
outlet	120	Brand, district, depot, dock_type, parking, delivery windows, usual vehicles
vehicle	60	Type, temp, capacity (weight/volume), km per litre, weekly fuel quota, depot
service_allowance	9	Handling minutes per (brand, dock_type)
calendar_day	910	Dates, day-of-week, holidays, monsoon flag
traffic_speed	576	Speed index per (district, hour, monsoon) - ML feature
product	11	SKU, brand, temperature, unit kg/m ³ , UOM
app_user	6	Email, role, scope (depot/dock/vehicle/outlet), scrypt password hash
workspace	1+	One per demo instance, clock columns, sandbox flag

4. Workspace (Operational) Tables

Table	Purpose
vehicle_day	Vehicle status and fuel-used for the service date
standing_order	Store's usual basket, placed automatically at the ordering cutoff
order_	One delivery order per outlet, temperature and service date
order_line	One SKU line, tracks qty from ordered through loaded to delivered and received
plan	One CP-SAT solve, holds KPIs, policy, status (draft/published/superseded)
trip	One vehicle trip (trip_no 1 or 2), status, dock bay, seal, signal state
stop	One order on one trip, sequence, planned and actual times, outcome
deferral	Why an order was not served (kind, code, cost)
issue	Dispatcher action item: short/damaged at dock, driver report, receipt issue
proof	Photo and signature for a completed stop
media	Binary photos and signatures stored as bytea
receipt	Store manager's confirmation of an order
pending_move	Dispatcher order-move queued while the target van has no signal
notice	One message in a role's notification feed
device_seen	Last heartbeat from a driver's phone (for dark-van detection)
event	Append-only audit log, also stores offline idempotency keys

5. Key Lifecycles

Only the states the code writes are shown, spelled as in the code. The functions are in these files:

domain/ordering.py: place_order, submit_basket

domain/planning.py: close_orders, persist, publish, move_order, _apply_move, defer_order

domain/fieldops.py: check_line, flag_line, decide_issue, release_trip, start_run, arrive, complete, report_problem, heartbeat, detect_dark, apply_pending_moves, confirm_receipt

domain/simulate.py: advance and _auto_receipts, which move the trips nobody is driving through the same states

Order (order_.status)

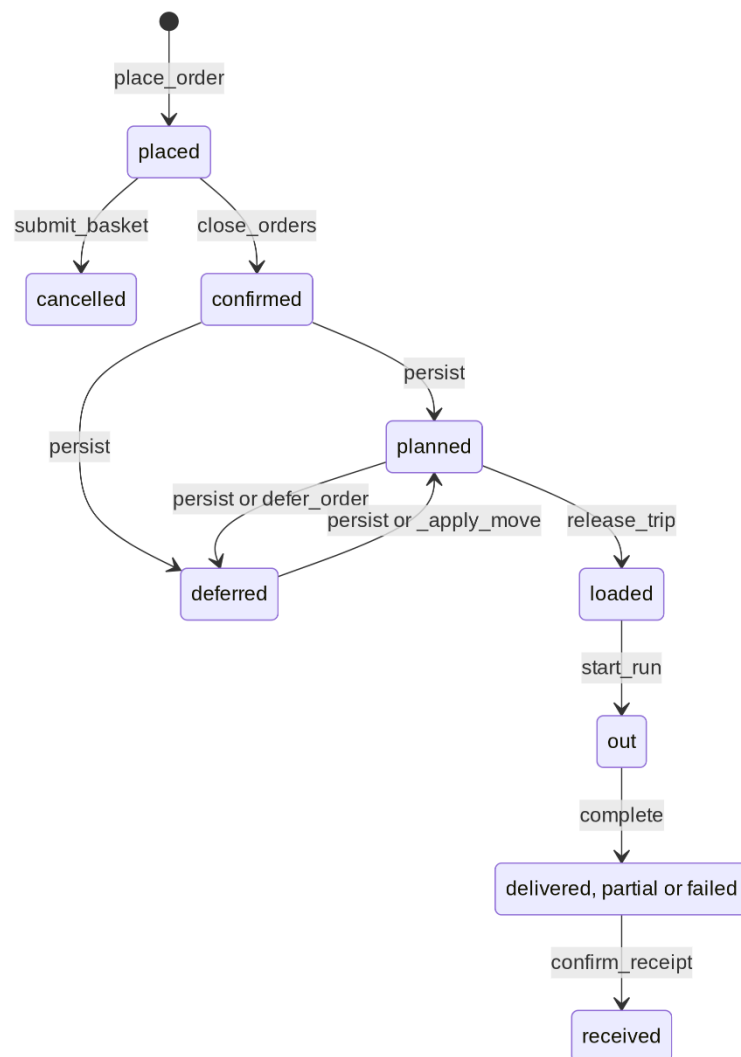


Figure 4: Order

5.1 Order Status

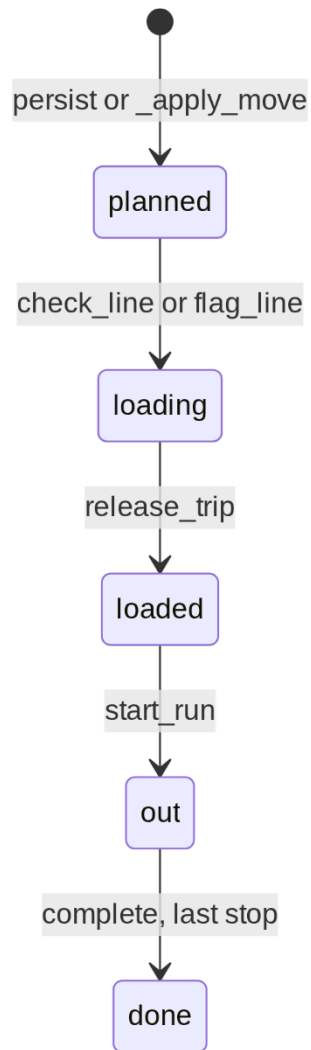


Figure 5: Trip

Status	Set By
placed	place_order (store submits basket or standing order fires)
confirmed	close_orders (4 PM cutoff)
planned	persist (assigned to a trip in the current plan)
deferred	persist or defer_order (not on any trip)
loaded	release_trip (dock releases the vehicle)
out	start_run (driver starts the run)
delivered / partial / failed	complete (driver records outcome at each stop)
received	confirm_receipt (store confirms) or auto-receipt

5.2 Trip Status

Status	Set By
planned	persist or _apply_move
loading	First dock action (check_line or flag_line)
loaded	release_trip (seal, temperature, doors check)
out	start_run
done	complete at the last stop

5.3 Issue Resolution

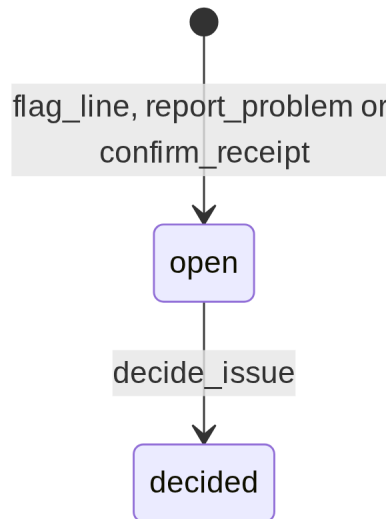


Figure 6: Issue

Issue Kind	Raised By	Decisions
short_at_dock / damaged_at_dock / wrong_at_dock	Loader via flag_line	partial · hold · next (follow-up order)
driver_report	Driver via report_problem	ack
receipt_issue	Store via confirm_receipt	ack

6. Event Log & Live Updates

record() (domain/events.py) is the only function that writes to the event table. It appends a row and runs pg_notify('relay_events', ...) in the same transaction. Postgres delivers the notification only on commit, so browsers never hear about a write that was rolled back. Event rows are never updated or deleted (except on workspace wipe). The event log also stores offline idempotency keys via the unique constraint uq_event_client_id(workspace_id, client_event_id).

7. Virtual Clock

Each workspace has its own virtual clock so a judge can walk through a full delivery day in minutes. The formula is:

$$\text{now} = \text{clock_anchor_virtual} + (\text{real_now} - \text{clock_anchor_real}) \times \text{clock_rate}$$

The demo starts paused at Tuesday 29 Sep 2026 15:20 - forty minutes before the 4 PM ordering cutoff for Wednesday. Field actions advance the clock automatically (e.g. releasing a trip jumps to 10 min before departure). All business code calls `now_virtual(ws)`; raw real time is used only for heartbeat tracking and cache headers.

8. Data Privacy & Security Rules

Rule	Implementation
Competition data not in git	.gitignore excludes data/private/ and all private CSVs
Media access-controlled	GET /api/media/{id} requires a valid session in the same workspace
Passwords as scrypt hashes	security.py; sessions are signed JWT cookies, not stored in DB
Workspace row isolation	Every workspace query filters on workspace_id; ON DELETE CASCADE
Media size limit	MAX_MEDIA_BYTES (default 2 MB); JPEG/PNG/WebP/SVG only